



Eventix

Event Ticketing Platform

Part 1 of 2 — **Product & Features Walkthrough**

Native Android app + backend — with race-condition-safe seat sales.

Kotlin

Jetpack Compose

Retrofit

Node + Express

PostgreSQL

JWT

ZXing / ML Kit

Prepared 14 July 2026 · Author: Bhanu Prakash

o • What Eventix is

A full-stack event-ticketing product: a native **Android app** (Kotlin + Jetpack Compose) for attendees and organizers, backed by a **Node/Express REST API** over PostgreSQL. Attendees discover events and buy a ticket in one tap; organizers create events and scan attendees in at the gate with the phone camera. The engineering showpiece is that a sold-out event *never* sells seat #51 — even under heavy concurrency.

Interviewer one-liner: "It's a Compose MVVM app talking to an Express+Postgres API. The headline is correctness under concurrency: seat purchase is a transaction with a pessimistic row lock, a DB-level oversell CHECK, and idempotency keys, so 200 people hitting a 50-seat event get exactly 50 tickets — proven by a load test."

LAYER	CHOICE
Android	Kotlin 2.0 · Jetpack Compose (Material 3) · MVVM (StateFlow) · manual DI (AppContainer) · Navigation-Compose
Networking	Retrofit + OkHttp + kotlinx-serialization · JWT in DataStore, injected by an OkHttp interceptor
QR	ZXing (generate on-device) · CameraX + ML Kit barcode (scan)
Backend	Node 20 (ESM) · Express 4 · raw SQL via <code>pg</code> (no ORM) · Zod validation
Database	PostgreSQL 17 — embedded (npm) on :5433 in dev, or Docker
Auth / crypto	JWT (7-day, HS256) · bcrypt · HMAC-SHA256 signed QR codes (timing-safe verify)

Note on screenshots: the Android UI needs a device or emulator, which wasn't available in this capture environment, so the app screens below are described from the code rather than shown. A pre-built debug APK ships in the repo (`android/app/build/outputs/apk/debug/app-debug.apk`). The **backend artifacts are real and captured live** — see the load-test and API response on the next page.

1 • Proof it works — the concurrency load test

The flagship feature is impossible to fake, so the repo ships an executable spec (`loadtest/oversell-test.js`) that fires 200 simultaneous buyers at a fresh 50-seat event and asserts every guarantee. Here is a real run:

```
Oversell test against http://localhost:3000
```

- ✓ logged in with seeded demo accounts
- ✓ created event #42 with capacity 50

```
Firing 200 concurrent purchase requests...
```

```
Done in 1730ms → 50 sold, 150 rejected as sold out, 0 errors
```

- ✓ no unexpected errors
- ✓ exactly 50 purchases succeeded (no overselling)
- ✓ event.ticketsSold === 50 in the database
- ✓ seatsLeft === 0
- ✓ all 50 ticket codes are unique
- ✓ sold-out event rejects a new purchase
- ✓ replaying an idempotency key returns the original ticket (no double sale)
- ✓ first QR scan checks the ticket in
- ✓ second scan of the same QR is rejected
- ✓ forged QR signature is rejected

🎉 All concurrency guarantees hold.

Live load-test: 200 concurrent buyers against a 50-seat event → exactly 50 sold, zero oversold.

A live API response

```
GET /api/events (Authorization: Bearer <jwt>)
{
  "events": [
    {
      "id": 7,
      "title": "Neon Nights – Live",
      "venue": "Load Test Arena",
      "capacity": 50,
      "ticketsSold": 50,
      "seatsLeft": 0,
      "priceCents": 1000,
      "currency": "INR",
      "organizerName": "Demo Organizer"
    }
  ]
}
```

Live API response: the server computes `seatsLeft` per event so the app can render "Sold out" / "Only N left" chips.

2 • The flagship — race-condition-safe ticket sales

Overselling is the classic ticketing bug: two requests both read "1 seat left" and both succeed. Eventix defends against it in **three independent layers**, so even a bug in one is caught by the next.

#	LAYER	WHAT IT DOES
1	Pessimistic row lock	The purchase transaction runs <code>BEGIN</code> → <code>SELECT ... FOR UPDATE</code> on the event row, serializing all concurrent buyers for that event so only one checks capacity at a time.
2	DB-level CHECK	The <code>events</code> table has <code>CHECK (tickets_sold <= capacity) (no_oversell)</code> — a last-resort guarantee the database itself enforces, independent of app code.
3	Idempotency key	Each purchase carries a client-generated <code>Idempotency-Key</code> (UUID) with a <code>UNIQUE</code> constraint, so a retried tap can never buy two tickets — a replay returns the original ticket.

A subtle real-world bug the code fixes: the purchase handler deliberately *reuses the same connection* it locked with for the follow-up read. An earlier version grabbed a second connection from the pool while holding the lock, which could deadlock under load (pool starvation). Reusing the one connection avoids it — a great "war story" talking point.

Check-in and anti-forgery

- **Atomic check-in:** scanning a QR runs a single conditional `UPDATE ... WHERE status='valid'`. If two gate staff scan the same code at once, exactly one update succeeds — the second is rejected as "already checked in".
- **Unforgeable QR:** each ticket's QR payload is signed with **HMAC-SHA256** (payload prefixed `ETP1.`). The server re-computes the signature with a **timing-safe comparison**; a forged or tampered code is rejected. QR codes are generated on-device (ZXing) and never stored as images.

3 • The Android app (screen by screen)

The app is 100% Kotlin + Jetpack Compose (no XML layouts), MVVM with one `ViewModel` per screen exposing a `StateFlow`, and a role-based navigation graph. Brand name: **Eventix**.

SCREEN	WHAT IT DOES
Splash	Brand-gradient splash while the saved session (JWT) loads.
Auth	One screen toggling sign-in / register; register has a role picker (Attendee vs Organizer segmented buttons), password show/hide, inline validation, demo-account hint.
Discover / Home	Searchable event list (by title or venue), pull-to-refresh, live seats-left chips (green / amber "Only N left" / red "Sold out"), price chips, cover images (Coil, Unsplash URLs), greeting header, logout.
Event Detail	Hero image, date/venue/host, an availability progress bar (sold vs capacity), description, sticky price + Buy ticket (idempotent), success dialog → "View ticket".
Tickets wallet	Purchased tickets as cards, each rendering an on-device QR (forced black-on-white), VALID / USED status chips, empty state.
Scanner (organizer)	CameraX live preview + ML Kit QR detection, camera-permission gate, in-flight guard (one check-in per code), colored result card ("Welcome, {name}!" vs "Already checked in / Wrong event / Not found / Invalid (forged)"), haptic feedback.
Create Event (organizer)	Form: title, venue, date picker, time, price (₹), capacity, optional cover URL, description; client-side validation; publishes to the API.

Role-based nav: attendees see Discover + Tickets; organizers additionally see Scan + Create. The bottom nav is built from the logged-in user's role, read from a `SessionManager` `StateFlow`.

The backend API

METHOD · PATH	AUTH	PURPOSE
GET /health	—	Liveness check
POST /auth/register · /auth/login	—	Create account (attendee/organizer) / authenticate → JWT
GET /events?q=	user	List/search events with computed <code>seatsLeft</code>
GET /events/:id	user	Event detail
POST /events	organizer	Create event
POST /events/:id/purchase	user + <code>Idempotency-Key</code>	Race-safe seat purchase (the flagship path)
GET /me/tickets	user	Ticket wallet (with signed QR payloads)
POST /checkin	organizer	Atomic QR check-in

4 · Likely interview questions

Q: How do you prevent overselling under concurrency?

Three layers: a pessimistic `SELECT ... FOR UPDATE` row lock serializes buyers per event, a DB `CHECK(tickets_sold <= capacity)` is the last line of defense, and a UNIQUE idempotency key stops double-buys on retry. A 200-buyer load test against 50 seats proves exactly 50 sell.

Q: What was the trickiest bug?

A pool-starvation deadlock: the purchase transaction held a row lock and then asked the pool for a *second* connection for a follow-up read. Under load the pool emptied and it deadlocked. Fix: reuse the same connection for the whole transaction.

Q: How are tickets made unforgeable?

The QR payload is HMAC-SHA256-signed server-side; check-in re-signs and compares with a timing-safe equal. A tampered payload fails the signature check and is rejected — no database lookup can be tricked.

Q: How does the app stay in sync with auth?

The JWT lives in DataStore; a `SessionManager` exposes a `StateFlow<SessionState>` that the root nav observes to switch between the Auth graph and the logged-in graph. An OkHttp interceptor attaches the token to every request.

Q: Why no ORM / no Hilt?

Deliberate, for explainability: raw parameterized SQL makes the locking and constraints visible; manual DI (an `AppContainer`) keeps the wiring readable in an interview.